Data Structures and Algorithms (DSA) Detailed Guide for PDF

What is DSA?

Data Structures and Algorithms (DSA) is a fundamental concept in computer science used to organize and process data efficiently.

Why Learn DSA?

- Improves problem-solving skills
- Helps write efficient code
- Important for interviews
- Used in real-world applications

Data Structures

What is a Data Structure?

A way of organizing and storing data for efficient access and modification.

Types of Data Structures

1. Linear Data Structures

- Array
- Linked List
- Stack
- Queue

2. Non-Linear Data Structures

- Tree
- Graph

Arrays

```
int arr[5] = {1,2,3,4,5};
```

- Fixed size
- Fast access

Linked List

- Dynamic size
- Nodes connected via pointers

Stack (LIFO)

Operations:

- Push
- Pop

Example:

```
stack.push(10);
```

Queue (FIFO)

Operations:

- Enqueue
- Dequeue

Trees

- Hierarchical structure
- Root, child, leaf nodes

Types:

- Binary Tree
- Binary Search Tree (BST)

Graphs

- Collection of nodes and edges
- Directed / Undirected

Algorithms

What is an Algorithm?

A step-by-step procedure to solve a problem.

Types of Algorithms

- Searching
- Sorting
- Recursion
- Dynamic Programming
- Greedy Algorithms

Searching Algorithms

Linear Search

```
for(i=0;i<n;i++){
 if(arr[i]==key) return i;
}
```

Binary Search

```
// Requires sorted array
```

Sorting Algorithms

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort

Recursion

Function calling itself.

```
int fact(int n) {
 if(n==0) return 1;
 return n * fact(n-1);
}
```

Time Complexity

Measures efficiency of algorithm.

Common notations:

- O(1)
- O(n)
- O(log n)
- O(n^2)

Space Complexity

Amount of memory used by algorithm.

Example

```
int sum = 0;
for(int i=0;i<n;i++){
 sum += i;
}
```

Time Complexity: O(n)

Advantages of DSA

- Efficient programs
- Better performance

Disadvantages

- Complex for beginners

Best Practices

- Practice regularly
- Understand logic
- Solve problems daily